

Отчёт по лабораторной работе №12

**Программирование в командном процессоре ОС UNIX.
Командные файлы**

Магос Иоаннис

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	9
4	Контрольные вопросы	10

Список иллюстраций

2.1	Первый скрипт	6
2.2	второй скрипт	7
2.3	третий скрипт	7
2.4	четвёртый скрипт	8

Список таблиц

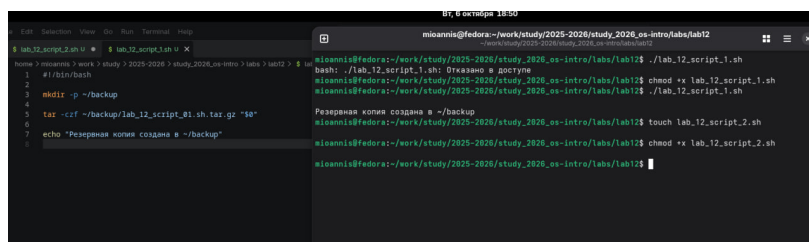
1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

2 Выполнение лабораторной работы

1. Выполняем первую задачу:

- Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.



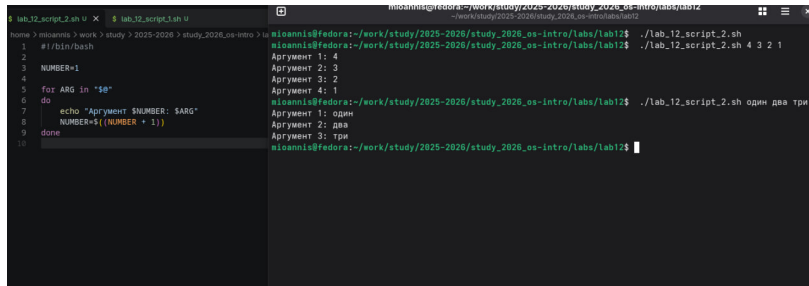
```
lab12_script_1.sh
1 #!/bin/bash
2
3 mkdir -p ~/backup
4
5 tar -czf ~/backup/lab12_script_01.sh.tar.gz "se"
6
7 echo "Резервная копия создана в ~/backup"
8
```

```
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$ ./lab12_script_1.sh
bash: ./lab12_script_1.sh: Permission denied
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$ chmod +x lab12_script_1.sh
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$ ./lab12_script_1.sh
Резервная копия создана в ~/backup
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$ touch lab12_script_2.sh
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$ chmod +x lab12_script_2.sh
miomannia@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab12$
```

Рисунок 2.1: Первый скрипт

2. Выполняем вторую задачу:

- Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.

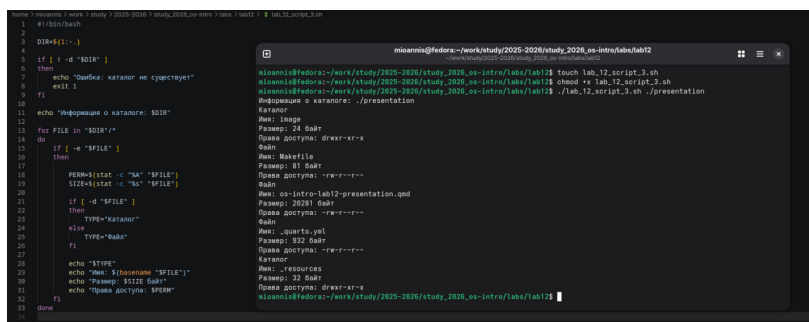


```
lab_12_script_2.sh v X $ lab_12_script_2.sh 4 3 2 1
1 #!/bin/bash
2
3 NUMBER=1
4
5 for ARG in "$@"
6 do
7     echo "Аргумент $NUMBER: $ARG"
8     NUMBER=$((NUMBER + 1))
9 done
10
```

Рисунок 2.2: второй скрипт

3. Выполняем третью задачу:

- Написать командный файл — аналог команды `ls` (без использования самой этой команды и команды `dir`). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.



```
1 #!/bin/bash
2
3 DIR=${1:-.}
4
5 if [ ! -d "$DIR" ]
6 then
7     echo "Ошибка: каталог не существует"
8     exit 1
9 fi
10
11 echo "Информация о каталоге: $DIR"
12
13 for FILE in "$DIR"/*
14 do
15     if [ -d "$FILE" ]
16     then
17         PERM=$(stat -c "%a" "$FILE")
18         SIZE=$(stat -c "%s" "$FILE")
19         if [ -d "$FILE" ]
20         then
21             TYPE="каталог"
22         else
23             TYPE="файл"
24         fi
25         echo "Имя: $FILE"
26         echo "Размер: $(stat -c "%s" "$FILE") байт"
27         echo "Права доступа: $PERM"
28     fi
29 done
```

Рисунок 2.3: третий скрипт

4. Выполняем четвёртую задачу:

- Написать командный файл, который получает в качестве аргумента командной строки формат файла (`.txt`, `.doc`, `.jpg`, `.pdf` и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

The screenshot shows a terminal window with a dark background. On the left, a shell script is being edited. The script starts with a shebang line, followed by a comment in Russian. It contains an if statement that checks if a variable is set, and an echo statement. Then, it checks if a directory exists, and if not, it echoes a message and exits. Finally, it enters a loop that iterates over files in a directory, counts them, and appends a suffix to the filename. On the right, a separate terminal window shows the execution of the script. The prompt is 'miaoan@fedora:~/work/study/2025-2026/study_2026_cs-intro/labs/lab12'. The command './lab_12_script_4.sh .' is entered, and the output shows the count of files and the resulting filename.

```
1 #!/bin/bash
2
3 if [ $# -ne 2 ]
4 then
5     echo "Использование: $0 путь_к_каталогу"
6     exit 1
7 fi
8
9 EXT=$1
10 DIR=$2
11
12 if [ ! -d "$DIR" ]
13 then
14     echo "Каталог не найден"
15     exit 1
16 fi
17
18 COUNT=0
19
20 for FILE in "$DIR"/*
21 do
22     if [ -f "$FILE" ]
23     then
24         FILE_EXT="${FILE##*.}"
25
26         if [ "$FILE_EXT" = "$EXT" ]
27         then
28             COUNT=$((COUNT + 1))
29         fi
30     fi
31 done
32
33 echo "Найдено $EXT файлов: $COUNT"
```

```
miaoan@fedora:~/work/study/2025-2026/study_2026_cs-intro/labs/lab12
miaoan@fedora:~/work/study/2025-2026/study_2026_cs-intro/labs/lab12$ ./lab_12_script_4.sh .
Найдено .sh файлов: 4
miaoan@fedora:~/work/study/2025-2026/study_2026_cs-intro/labs/lab12$
```

Рисунок 2.4: четвёртый скрипт

3 Вывод

В данной работе мы изучили основы программирования в оболочке ОС UNIX/Linux. Научились писать небольшие командные файлы и скрипты на языке `bush`.

4 Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются? Ответ:
 - sh — стандартная командная оболочка UNIX/Linux, содержащая базовый, полный набор функций
 - csh — использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд
 - ksh — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна
 - bash — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна
2. Что такое POSIX? Ответ: POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ.
3. Как определяются переменные и массивы в языке программирования bash? Ответ: Переменные вызываются `$var`, где `var`=чему-то, указанному пользователем, неважно что бы то не было, название файла, каталога или еще чего. Для массивов используется команда `set -A`
4. Каково назначение операторов `let` и `read`? Ответ: `let` — вычисляет далее заданное математическое значение `read` — позволяет читать значения переменных со стандартного ввода

5. Какие арифметические операции можно применять в языке программирования bash? Ответ: Прибавление, умножение, вычисление, деление, сравнение значений, экспонирование и др.
6. Что означает операция (())? Ответ: Это обозначение используется для облегчения программирования для условий bash
7. Какие стандартные имена переменных Вам известны? Ответ: Нам известны HOME, PATH, BASH, ENV, PWD, UID, OLDPWD, PPID, GROUPS, OSTYPE, PS1 - PS4, LANG, HOSTFILE, MAIL, TERM, LOGNAME, USERNAME, IFS и др.
8. Что такое метасимволы? Ответ: Метасимволы это специальные знаки, которые могут использоваться для сокращения пути, поиска объекта по расширению, перед переменными, например «\$» или «*» .
9. Как экранировать метасимволы? Ответ: Добавить перед метасимволом метасимвол «\»
10. Как создавать и запускать командные файлы? Ответ: При помощи команды chmod надо дать права на запуск (chmod +x название файла), затем запустить bash полный/путь/к/файлу.sh Например у нас файл lab Пишем: chmod +x lab ./lab
11. Как определяются функции в языке программирования bash? Ответ: Объединяя несколько команд с помощью function
12. Каким образом можно выяснить, является файл каталогом или обычным файлом? Ответ: Можно задать команду на проверку директория ли это test -d директория
13. Каково назначение команд set, typeset и unset? Ответ: Set — используется для создания массивов Unset — используется для изъятия переменной Typeset — используется для присваивания каких-либо функций

14. Как передаются параметры в командные файлы? Ответ: Добавлением аргументов после команды запуска `bash` скрипта

15. Назовите специальные переменные языка `bash` и их назначение. Ответ:

- `$*` – отображается вся командная строка или параметры оболочки;
- `$?` – код завершения последней выполненной команды;
- `$$` – уникальный идентификатор процесса, в рамках которого выполняется командный файл;
- `#!` – номер процесса, в рамках которого выполняется последняя вызванная на выполнение команда;
- `$-` – значение флагов командного процессора;
- `${#*}` – возвращает целое число – количество слов, которые были результатом `$*`;
- `${#name}` – возвращает целое значение длины строки в переменной `name`;
- `{name[n]}` – обращение к `n`-му элементу массива;
- `{name[*]}` – перечисляет все элементы массива, разделённые пробелом;
- `{name[@]}` – то же самое, но позволяет учитывать символы пробелы в самих переменных;
- `{name:-value}` – если значение переменной `name` не определено, то оно будет заменено на указанный `value`;
- `{name:value}` – проверяется факт существования переменной;
- `{name=value}` – если `name` не определено, то ему присваивается значение `value`;
- `{name?value}` – останавливает выполнение, если имя переменной не определено, и выводит сообщение об ошибке;
- `{name+value}` – это выражение работает противоположно `{name=value}`. Если переменная определена, то подставляется `value`;
- `{name#pattern}` – представляет значение переменной `name` с удалённым самым коротким подстроком, соответствующим `pattern`;
- `${#name[*]}` и `${#name[@]}` – эти выражения возвращают количество элементов в массиве `name`.